ChatGPT gpt-5-3

👤 DEV 2 — PLAN (UPDATED: SynapseSignal Traffic Intelligence Layer with Flow-Aware Modeling)

🔷 ROLE

Convert perception outputs into **flow-aware traffic intelligence** supporting:

- **Intersection-level decisions (real-time)**
- **City-level optimization (graph-based)**
- **Downstream-aware congestion control (NEW)**

Dev 2 ensures:

- Accurate tracking
- Lane-level modeling
- Downstream capacity awareness
- Sector-wise aggregation
- Clean, structured output for Dev 3

🔷 PHASED EXECUTION PLAN (UPDATED)

🔹 PHASE 1 — DETECTION OUTPUT INGESTION

Goal: Standardize Dev 1 outputs

Tasks:

- Parse:
  - bbox
  - class
  - type
- Add:
  - frame_id
  - timestamp

- Convert to pixel coordinates

Deliverable:
✔ Standardized Detection Objects

🔹 PHASE 2 — MULTI-OBJECT TRACKING (BYTETRACK)

Goal: Maintain identity across frames

Tasks:

- Assign:
  - object_id
- Track:
  - trajectory
  - velocity
- Handle:
  - occlusion
  - re-identification

Deliverable:
✔ Tracking Output

🔹 PHASE 3 — LANE-LEVEL MAPPING (NEW CRITICAL PHASE)

Goal: Map vehicles to lanes

Tasks:

- Define:
  - lane regions (ROI per lane)
- Assign each tracked object → lane_id
- Maintain:
  - lane-wise vehicle lists

Deliverable:
✔ Lane-wise Vehicle Mapping

🔹 PHASE 4 — LANE-LEVEL FEATURE EXTRACTION

Goal: Compute per-lane traffic metrics

Tasks:
For each lane:

- Compute:
  - in_density (vehicles waiting)
  - avg_speed
  - queue_length

Deliverable:
✔ Lane-Level Metrics

🔹 PHASE 5 — DOWNSTREAM STATE ESTIMATION (NEW CORE LOGIC)

Goal: Model outgoing lane conditions

Tasks:

- Map:

```text
incoming_lane → outgoing_lane
```

- Compute:
  - out_density
  - estimated capacity
- Use:
  - downstream intersection data OR heuristic

Deliverable:
✔ Downstream Lane State

🔹 PHASE 6 — FLOW READINESS FEATURE COMPUTATION

Goal: Prepare inputs for flow optimization

Tasks:
For each lane:

```text
flow_score_inputs = {
  in_density,
  out_density,
```

```
  capacity
}
```

Deliverable:
✔ Flow-Aware Lane Features

🔹 PHASE 7 — SECTOR-WISE AGGREGATION (NEW)

Goal: Group lanes into sectors

Tasks:

- Define sectors:
  - North-South
  - East-West
  - Turning lanes
- Aggregate:
  - density
  - flow inputs

Deliverable:
✔ Sector-Level Traffic Representation

🔹 PHASE 8 — INTERSECTION TRAFFIC STATE MODELING

Goal: Build complete intersection state

Tasks:
Combine:

- lane-level metrics
- downstream state
- sector data
- emergency state

Output:

```json
{
  "intersection_id": "...",
  "lanes": [...],
  "sectors": [...],
```

```
  "emergency_state": {...}
}
```

Deliverable:
✔ Flow-Aware Intersection State

🔹 PHASE 9 — GRAPH CONSTRUCTION (CITY LEVEL)

Goal: Represent city as graph

Tasks:

- Nodes → intersections
- Edges → roads
- Attach:
  - flow metrics
  - densities

Deliverable:
✔ Traffic Graph

🔹 PHASE 10 — CITY-LEVEL STATE AGGREGATION

Goal: Prepare input for global model

Tasks:

- Aggregate:
  - all intersection states
- Normalize data

Deliverable:
✔ City-Level Traffic State

🔹 PHASE 11 — OUTPUT API FOR DEV 3 (UPDATED)

Goal: Provide complete decision-ready data

Output format:

```json
{
  "intersection_id": "...",
```

```
  "lanes": [
    {
      "lane_id": "...",
      "in_density": ...,
      "out_density": ...,
      "capacity": ...
    }
  ],
  "sectors": [...],
  "emergency_state": {...},
  "city_state": {...}
}
```

Deliverable:
✔ Flow-Aware API

🔷 DEV 2 SUCCESS CRITERIA (UPDATED)

- Stable multi-object tracking
- Accurate lane-level mapping
- Correct in_density computation
- Reliable downstream estimation
- Flow-ready features computed
- Sector-wise aggregation working
- Clean intersection state representation
- Graph-based city modeling ready
- API supports flow optimization logic

🧠 FINAL INSIGHT

> Dev 2 no longer just measures traffic —
> it **models how traffic can flow**

✔ Output now fully supports
👉 **Dev 3 Flow Optimization + Sector-Based Signal Control**

ChatGPT gpt-5-3

👤 DEV 2 — MASTER PROMPTS (UPDATED: Flow-Aware Traffic Intelligence | SynapseSignal)

1️⃣ MASTER PROMPT — DETECTION INGESTION & STANDARDIZATION

You are a computer vision systems engineer.

Task:
Standardize Dev 1 output into structured input for tracking.

Input:

- bbox (normalized)
- class
- type
- confidence

You must:

- Add:
  - frame_id
  - timestamp
- Convert:
  - normalized → pixel coordinates
- Ensure schema consistency

Output format:

```json
{
  "frame_id": "...",
  "timestamp": "...",
  "objects": [
    {
      "object_id": null,
      "class": "...",
      "type": "...",
      "bbox_pixel": [...],
      "confidence": ...
    }
  ]
```

```
}
```

Constraints:

- No missing fields
- No inconsistent formats

2️⃣ MASTER PROMPT — MULTI-OBJECT TRACKING (BYTETRACK)

You are implementing a tracking system.

Task:
Track objects across frames with stable IDs.

You must:

- Assign:
  - unique object_id
- Maintain:
  - identity across frames
- Compute:
  - velocity
  - trajectory

Output format:

```json
{
  "object_id": "...",
  "trajectory": [...],
  "velocity": ...
}
```

Constraints:

- No ID switching
- No duplicate IDs

3️⃣ MASTER PROMPT — LANE-LEVEL MAPPING (CRITICAL)

You are designing spatial mapping logic.

Task:
Assign each tracked vehicle to a lane.

You must:

- Define lane regions (ROI polygons)
- Map object centroid → lane_id
- Maintain:
  - lane-wise object lists

Output format:

```json
{
  "lane_id": "...",
  "vehicles": ["object_id_1", "object_id_2"]
}
```

Constraints:

- No ambiguous lane assignment
- Must handle boundary cases

4️⃣ MASTER PROMPT — LANE-LEVEL FEATURE EXTRACTION

You are building traffic metrics.

Task:
Compute metrics per lane.

You must compute:

- in_density
- avg_speed
- queue_length

Output format:

```json
{
  "lane_id": "...",
  "in_density": ...,
  "avg_speed": ...,
  "queue_length": ...
```

```
}
```

Constraints:

- Must derive from tracking data
- No hardcoded values

5️⃣ MASTER PROMPT — DOWNSTREAM LANE MAPPING

You are modeling traffic flow direction.

Task:
Map incoming lanes to outgoing lanes.

You must:

- Define:

```text
incoming_lane → outgoing_lane
```

- Use:
  - road topology
  - intersection layout

Output format:

```json
{
  "incoming_lane": "...",
  "outgoing_lane": "..."
}
```

Constraints:

- Must reflect real traffic flow
- No disconnected mapping

6️⃣ MASTER PROMPT — DOWNSTREAM STATE ESTIMATION

You are modeling downstream capacity.

Task:
Compute outgoing lane state.

You must compute:

- out_density
- capacity

Output format:

```json
{
  "lane_id": "...",
  "out_density": ...,
  "capacity": ...
}
```

Constraints:

- Must use real data OR justified estimation
- No arbitrary constants

7️⃣ MASTER PROMPT — FLOW FEATURE PREPARATION

You are preparing inputs for control logic.

Task:
Prepare flow-related features.

You must output:

```json
{
  "lane_id": "...",
  "in_density": ...,
  "out_density": ...,
  "capacity": ...
}
```

Constraints:

- Must be complete for all lanes
- No missing values

8️⃣ MASTER PROMPT — SECTOR-WISE AGGREGATION

You are designing higher-level grouping.

Task:
Group lanes into sectors.

You must:

- Define sectors:
  - NORTH_SOUTH
  - EAST_WEST
- Aggregate:
  - density
  - flow inputs

Output format:

```json
{
  "sector_id": "...",
  "lanes": [...],
  "aggregated_density": ...
}
```

Constraints:

- No overlapping sectors
- Must cover all lanes

9️⃣ MASTER PROMPT — INTERSECTION STATE MODEL (UPDATED)

You are building unified traffic state.

Task:
Combine all lane + sector data.

Output format:

```json
{
  "intersection_id": "...",
  "lanes": [...],
```

```
  "sectors": [...],
  "emergency_state": {...}
}
```

Constraints:

- Must include all dependencies
- Must be consistent

🔟 MASTER PROMPT — CITY-LEVEL GRAPH CONSTRUCTION

You are modeling the entire city.

Task:
Create graph representation.

You must:

- Define:
  - nodes → intersections
  - edges → roads
- Attach:
  - flow metrics

Output format:

```json
{
  "nodes": [...],
  "edges": [...]
}
```

Constraints:

- Must be connected graph
- No missing links

1️⃣1️⃣ MASTER PROMPT — CITY STATE AGGREGATION

You are preparing input for global model.

Task:
Aggregate all intersection states.

Output format:

```
json

{
  "city_state": {
    "intersections": [...],
    "edges": [...]
  }
}
```

Constraints:

- Must be normalized
- No missing intersections

1️⃣2️⃣ MASTER PROMPT — OUTPUT API FOR DEV 3 (FINAL)

You are defining interface to control layer.

Task:
Provide full flow-aware traffic state.

Output format:

```
json

{
  "intersection_id": "...",
  "lanes": [
    {
      "lane_id": "...",
      "in_density": ...,
      "out_density": ...,
      "capacity": ...
    }
  ],
  "sectors": [...],
  "emergency_state": {...},
  "city_state": {...}
}
```

Constraints:

- Must be low-latency
- Must be consistent across frames

🔷 DEV 2 MASTER PROMPT OUTCOME (UPDATED)

- Clean ingestion pipeline
- Stable tracking system
- Accurate lane-level mapping
- Downstream-aware modeling
- Flow-ready feature generation
- Sector-based aggregation
- Complete intersection state
- Graph-based city representation
- Aggregated city-level state
- API ready for Dev 3 flow optimization

✔ SynapseSignal Intelligence Layer is now **flow-aware and decision-ready**

ChatGPT gpt-5-3

👤 DEV 2 — GUIDED LEARNING (UPDATED: Flow-Aware Traffic Intelligence | SynapseSignal)

🔹 PHASE 1 — DETECTION INGESTION

🧠 What You Are Learning

You are learning **how to convert perception into structured system input**.

⚙️ What Is Happening

- Dev 1 outputs detections
- You standardize:
  - coordinates
  - timestamps
  - schema

🧪 Guided Prompt

Run Detection Ingestion Prompt

🔍 What to Notice

- Missing timestamp → no temporal reasoning
- Inconsistent format → pipeline breaks

🧱 Mental Model

ML output must be structured before use

🔹 PHASE 2 — MULTI-OBJECT TRACKING

🧠 What You Are Learning

You are learning **object identity across time**.

⚙️ What Is Happening

- Assign persistent IDs
- Track vehicle movement

🧪 Guided Prompt

Run ByteTrack Prompt

🔍 What to Notice

- ID switching → corrupts speed/density
- Tracking enables motion understanding

🧱 Mental Model

Tracking = continuity of objects

🔹 PHASE 3 — LANE-LEVEL MAPPING (NEW CORE)

🧠 What You Are Learning

You are learning **spatial localization within intersections**.

⚙️ What Is Happening

- Define lane regions
- Assign each vehicle → lane

🧪 Guided Prompt

Run Lane Mapping Prompt

🔍 What to Notice

- Wrong lane → wrong decision later
- Boundary conditions are tricky

🧱 Mental Model

Lane = unit of traffic control

🔹 PHASE 4 — LANE-LEVEL FEATURE EXTRACTION

🧠 What You Are Learning

You are learning **fine-grained traffic measurement**.

⚙️ What Is Happening

For each lane:

- Count vehicles
- Compute speed
- Estimate queue

🧪 Guided Prompt

Run Lane Metrics Prompt

🔍 What to Notice

- Per-lane metrics > global metrics
- Needed for precise control

🧱 Mental Model

Granularity improves intelligence

🔹 PHASE 5 — DOWNSTREAM LANE MAPPING

🧠 What You Are Learning

You are learning **traffic flow direction modeling**.

⚙️ What Is Happening

- Map:

```text
incoming_lane → outgoing_lane
```

🧪 Guided Prompt

Run Lane Mapping Graph Prompt

🔍 What to Notice

- Wrong mapping → wrong flow decisions
- Depends on road topology

🧱 Mental Model

Traffic always flows somewhere

🔹 PHASE 6 — DOWNSTREAM STATE ESTIMATION (CRITICAL)

🧠 What You Are Learning

You are learning **flow feasibility modeling**.

⚙️ What Is Happening

- Measure outgoing lane:
  - density
  - capacity

🧪 Guided Prompt

Run Downstream Estimation Prompt

🔍 What to Notice

- High out_density → blocked exit
- Sending vehicles there → congestion

🧱 Mental Model

You must know where vehicles can go

🔹 PHASE 7 — FLOW FEATURE PREPARATION

🧠 What You Are Learning

You are learning **decision-ready feature engineering**.

⚙️ What Is Happening

Prepare:

```text
in_density + out_density + capacity
```

🧪 Guided Prompt

Run Flow Feature Prompt

🔍 What to Notice

- These features directly drive signal logic

- Missing values → bad decisions

🧱 Mental Model

Features define decisions

🔹 PHASE 8 — SECTOR-WISE AGGREGATION

🧠 What You Are Learning

You are learning **group-level traffic control modeling**.

⚙️ What Is Happening

- Group lanes:
  - North-South
  - East-West
- Aggregate metrics

🧪 Guided Prompt

Run Sector Aggregation Prompt

🔍 What to Notice

- Sector > individual lane in control systems
- Helps simplify decisions

🧱 Mental Model

Control happens at group level

🔹 PHASE 9 — INTERSECTION STATE MODEL

🧠 What You Are Learning

You are learning **complete system abstraction**.

⚙️ What Is Happening

- Combine:
  - lanes
  - sectors
  - emergency state

🧪 Guided Prompt

Run Intersection State Prompt

🔍 What to Notice

- Missing info → incomplete state
- Clean structure → easy decision making

🧱 Mental Model

State = full representation of intersection

🔹 PHASE 10 — CITY GRAPH CONSTRUCTION

🧠 What You Are Learning

You are learning **network-level modeling**.

⚙️ What Is Happening

- Intersections → nodes
- Roads → edges

🧪 Guided Prompt

Run Graph Construction Prompt

🔍 What to Notice

- Connectivity matters
- Wrong graph → wrong global optimization

🧱 Mental Model

City = graph of flows

🔹 PHASE 11 — CITY STATE AGGREGATION

🧠 What You Are Learning

You are learning **global state formation**.

⚙️ What Is Happening

- Combine all intersections
- Normalize data

🧪 Guided Prompt

Run City Aggregation Prompt

🔍 What to Notice

- Missing node → incomplete system
- Standardization is critical

🧱 Mental Model

Global intelligence = unified local states

🔹 PHASE 12 — OUTPUT API DESIGN

🧠 What You Are Learning

You are learning **system interface engineering**.

⚙️ What Is Happening

- Package all intelligence
- Send to Dev 3

🧪 Guided Prompt

Run API Prompt

🔍 What to Notice

- Clean API → smooth integration
- Bad schema → system failure

🧱 Mental Model

API = bridge between intelligence and control

🔷 FINAL LEARNING OUTCOME

You now understand:

- How detection becomes tracking
- How tracking becomes lane-level metrics
- How lanes map to traffic flow
- How downstream state affects decisions
- How to engineer flow-aware features
- How sectors simplify control
- How full intersection state is built
- How city is modeled as a graph
- How global state is constructed
- How to design system APIs

🧠 FINAL INSIGHT

> You are no longer measuring traffic —
> you are **modeling how traffic moves and where it can go**

✔ Ready for **Dev 3 (Flow-Aware Control Layer)**